Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему: «Бинарное дерево поиска»

Выполнили ст.гр. 25ВВВ3:
Волков И.Р.
Итянина В.А.
Приняли:
к.т.н., доцент Юрова О.В.
к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить бинарное дерево поиска и реализовать основные операции: построение, вывод, поиск элемента, подсчёт вхождений, исключение дубликатов и оценку сложности поиска.

Ход работы

Задание 1: реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#define _CRT_NONSTDC_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root;

struct Node* CreateTree(struct Node* root, int data) {
    if (root == NULL) {
        root = (struct Node*)malloc(sizeof(struct Node));
        if (root == NULL) {
            printf("Ошибка выделения памяти\n");
            exit(1);
        }
        root->data = data;
        root->left = NULL;
        root->right = NULL;
        return root;
    }

    if (data < root->data)
        root->left = CreateTree(root->left, data);
    else
        root->right = CreateTree(root->right, data);

    return root;
}

void print_tree(struct Node* r, int l) {
    if (r == NULL) return;

    print_tree(r->right, l + 1);

    for (int i = 0; i < l; i++)
```

```
        printf("    ");

    printf("%d\n", r->data);

    print_tree(r->left, l + 1);
}

struct Node* Search(struct Node* r, int data) {
    if (r == NULL) return NULL;
    if (r->data == data) return r;

    if (data < r->data)
        return Search(r->left, data);
    else
        return Search(r->right, data);
}

int main() {
    setlocale(LC_ALL, "");

    int D;
    int start = 1;

    root = NULL;

    printf("-1 - окончание построения дерева\n");

    while (start) {
        printf("Введите число: ");
        scanf_s("%d", &D);

        if (D == -1) {
            printf("Построение дерева окончено\n\n");
            start = 0;
        }
        else {
            root = CreateTree(root, D);
        }
    }

    printf("Дерево:\n");
    print_tree(root, 0);

    printf("\nВведите число для поиска: ");
    scanf_s("%d", &D);

    if (Search(root, D) != NULL)
        printf("Элемент %d найден в дереве.\n", D);
    else
        printf("Элемент %d не найден в дереве.\n", D);

    return 0;
}
```

Результат работы программы представлен на рисунках 1.1 и 1.2.

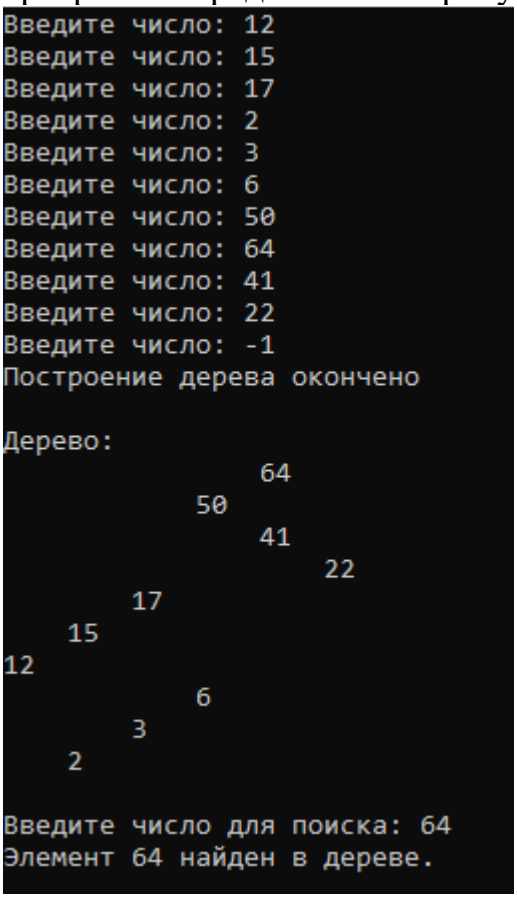


Рис.1.1 – результат работы программы к заданию 1.

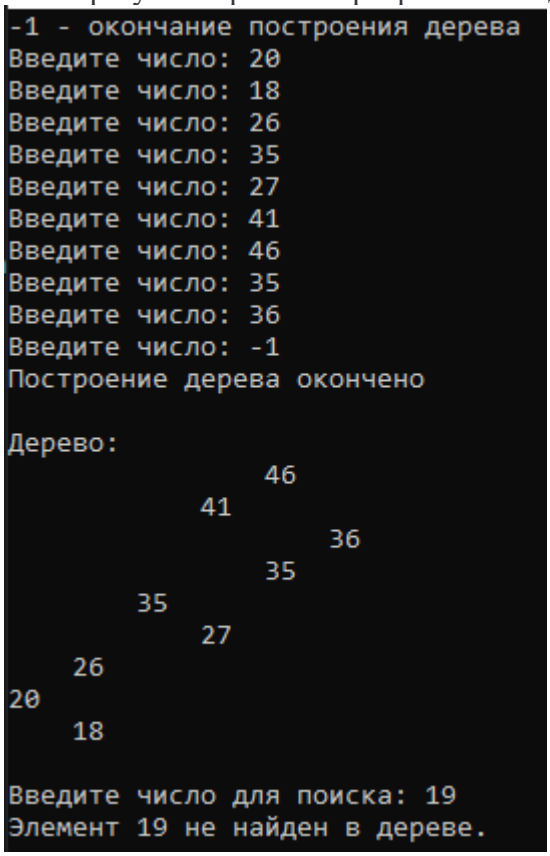


Рис.1.2 – результат работы программы к заданию 1.

Задание 2: реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#define _CRT_NONSTDC_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct Node
{
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root;

struct Node* CreateTree(struct Node* r, int data)
{
    if (r == NULL)
    {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL)
        {
            printf("Ошибка выделения памяти\n");
            exit(1);
        }
        r->data = data;
        r->left = NULL;
        r->right = NULL;
        return r;
    }

    if (data > r->data)
        r->left = CreateTree(r->left, data);
    else
        r->right = CreateTree(r->right, data);

    return r;
}

void print_tree(struct Node* r, int l)
{
    if (r == NULL)
        return;

    print_tree(r->right, l + 1);
```

```
    for (int i = 0; i < l; i++)
        printf("    ");

    printf("%d\n", r->data);

    print_tree(r->left, l + 1);
}

int Count(struct Node* r, int data)
{
    if (r == NULL)
        return 0;

    int count = (r->data == data) ? 1 : 0;

    return count + Count(r->left, data) + Count(r->right, data);
}

int main()
{
    setlocale(LC_ALL, "");

    int D;
    int start = 1;

    root = NULL;

    printf("-1 - окончание построения дерева\n");

    while (start)
    {
        printf("Введите число: ");
        if (scanf_s("%d", &D) != 1)
        {
            printf("Некорректный ввод, попробуйте снова.\n");
            while (getchar() != '\n');
            continue;
        }

        if (D == -1)
        {
            printf("Построение дерева окончено\n\n");
            start = 0;
        }
        else
        {
            root = CreateTree(root, D);
        }
    }

    printf("Дерево:\n");
    print_tree(root, 0);
```

```
    printf("\nВведите число для подсчёта: ");
    scanf_s("%d", &D);

    printf("Количество вхождений элемента %d: %d\n", D,
Count(root, D));

    return 0;
}
```

Результат работы программы представлен на рисунке 2.

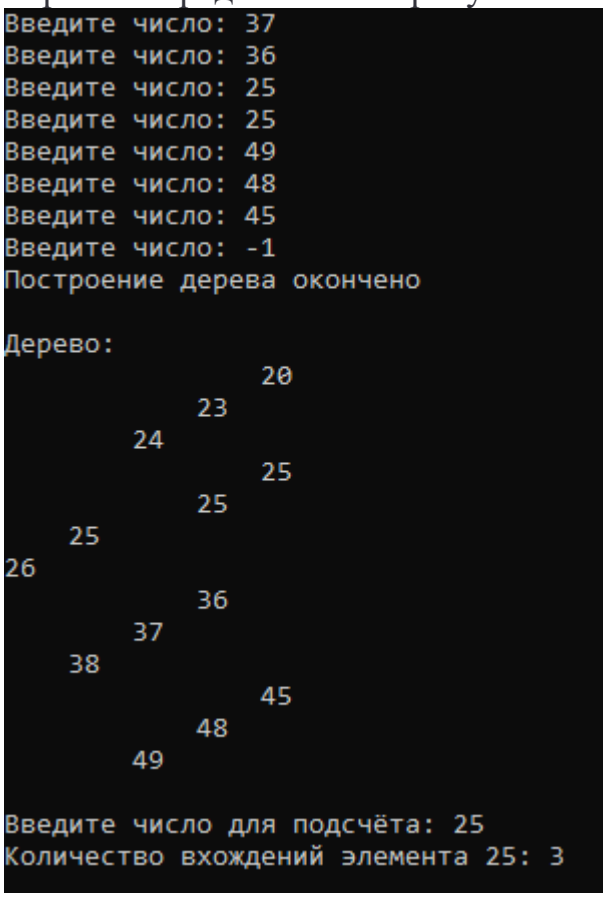


Рис.2 – результат работы программы к заданию 2.

Задание 3: изменить функцию добавления элементов для исключения добавления одинаковых символов.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#define _CRT_NONSTDC_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct Node
{
```

```
        int data;
        struct Node* left;
        struct Node* right;
    };

    struct Node* root;

    struct Node* CreateTree(struct Node* r, int data)
    {
        if (r == NULL)
        {
            r = (struct Node*)malloc(sizeof(struct Node));
            if (r == NULL)
            {
                printf("Ошибка выделения памяти\n");
                exit(1);
            }
            r->data = data;
            r->left = NULL;
            r->right = NULL;
            return r;
        }

        if (r->data == data)
        {
            printf("Число %d уже есть в дереве. Добавление
пропущено.\n", data);
            return r;
        }

        if (data > r->data)
            r->left = CreateTree(r->left, data);
        else
            r->right = CreateTree(r->right, data);

        return r;
    }

    void print_tree(struct Node* r, int l)
    {
        if (r == NULL) return;

        print_tree(r->right, l + 1);

        for (int i = 0; i < l; i++)
            printf("    ");

        printf("%d\n", r->data);

        print_tree(r->left, l + 1);
    }

    int main()
```

```
{
    setlocale(LC_ALL, "");

    int D;
    int start = 1;
    root = NULL;

    printf("-1 - окончание построения дерева\n");
    printf("Одинаковые числа добавляться не будут.\n\n");

    while (start)
    {
        printf("Введите число: ");
        scanf_s("%d", &D);

        if (D == -1)
        {
            printf("\nПостроение дерева окончено.\n\n");
            start = 0;
        }
        else
        {
            root = CreateTree(root, D);
        }
    }

    printf("Полученное дерево:\n");
    print_tree(root, 0);

    return 0;
}
```

Результат работы программы представлен на рисунке 3.

```
Введите число: 26
Введите число: 29
Введите число: 23
Число 23 уже есть в дереве. Добавление пропущено.
Введите число: 55
Введите число: 22
Введите число: 66
Введите число: 77
Введите число: 29
Число 29 уже есть в дереве. Добавление пропущено.
Введите число: -1

Построение дерева окончено.

Полученное дерево:
    22
23
    25
                26
                    29
            38
        45
            47
                52
                    55
                        66
                            77
```

Рис.3 – результат работы программы к заданию 3.

Задание 4: оценить сложность процедуры поиска по значению в бинарном дереве.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#define _CRT_NONSTDC_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct Node
{
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

struct Node* CreateTree(struct Node* r, int data)
{
    if (r == NULL)
    {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL)
        {
            printf("Ошибка выделения памяти\n");
            exit(1);
        }
        r->data = data;
        r->left = NULL;
        r->right = NULL;
        return r;
    }

    if (data > r->data)
        r->left = CreateTree(r->left, data);
    else
        r->right = CreateTree(r->right, data);

    return r;
}

void print_tree(struct Node* r, int l)
{
    if (r == NULL) return;

    print_tree(r->right, l + 1);
```

```
    for (int i = 0; i < l; i++)
        printf("    ");

    printf("%d\n", r->data);

    print_tree(r->left, l + 1);
}

struct Node* Search(struct Node* r, int data, int* steps)
{
    if (r == NULL)
        return NULL;

    (*steps)++;

    if (r->data == data)
        return r;

    if (data > r->data)
        return Search(r->left, data, steps);
    else
        return Search(r->right, data, steps);
}

int main()
{
    setlocale(LC_ALL, "");

    int D;
    int start = 1;
    root = NULL;

    printf("-1 - окончание построения дерева\n");

    while (start)
    {
        printf("Введите число: ");
        scanf_s("%d", &D);

        if (D == -1)
        {
            printf("\nПостроение дерева окончено.\n\n");
            start = 0;
        }
        else
        {
            root = CreateTree(root, D);
        }
    }

    printf("Полученное дерево:\n");
    print_tree(root, 0);
```

```
    printf("\nВведите число для поиска: ");
    scanf_s("%d", &D);

    int steps = 0;
    struct Node* result = Search(root, D, &steps);

    if (result != NULL)
        printf("Элемент %d найден.\n", D);
    else
        printf("Элемент %d не найден.\n", D);

    printf("Количество проверенных узлов: %d\n", steps);

    printf("\nОценка сложности поиска:\n");
    printf("В среднем случае: O(log n)\n");
    printf("В худшем случае: O(n)\n");

    return 0;
}
```

Результат работы программы представлен на рисунке 4.

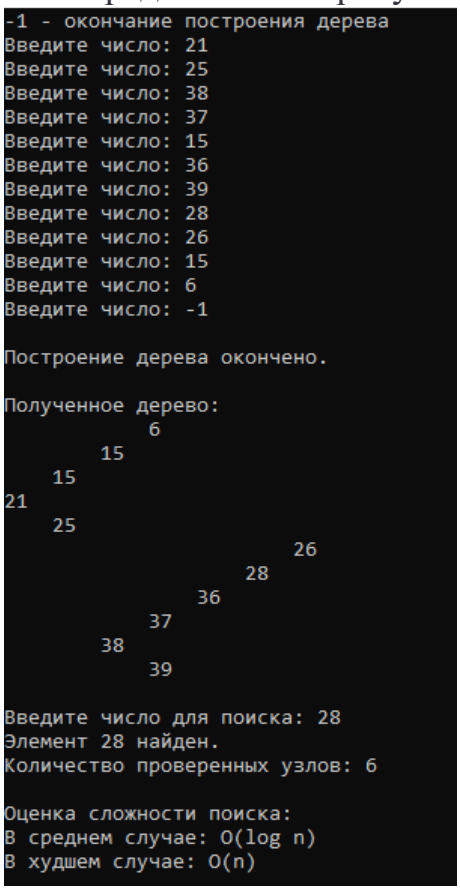


Рис.4 – результат работы программы к заданию 4.

Вывод: в ходе работы на языке С реализовано бинарное дерево поиска и все required операции: вставка, вывод, поиск, подсчёт вхождений и запрет дубликатов.